Virtual SQL Server Engine (VSSE) — A New Architectural Pattern

Authors

Hexagon Web Framework — Schema Builder & SQL Executor teams

Status

Concept Paper — July 1, 2026

---

1. Abstract

We define the **Virtual SQL Server Engine (VSSE)** pattern: a software architecture where SQL schema design, DDL generation, query building, syntax validation, dependency analysis, and execution planning operate **without a live database connection**. The database engine is an *abstract target* expressed through configuration, type maps, and dialect rules — not a TCP socket. Execution against a real database is deferred until explicitly requested, making the offline path the primary mode and the live path an optional deployment step.

This paper describes how the Hexagon Web Framework's Schema Builder and SQL Executor implement VSSE, and argues that this pattern represents a new category of database tooling.

---

2. The Problem That VSSE Solves

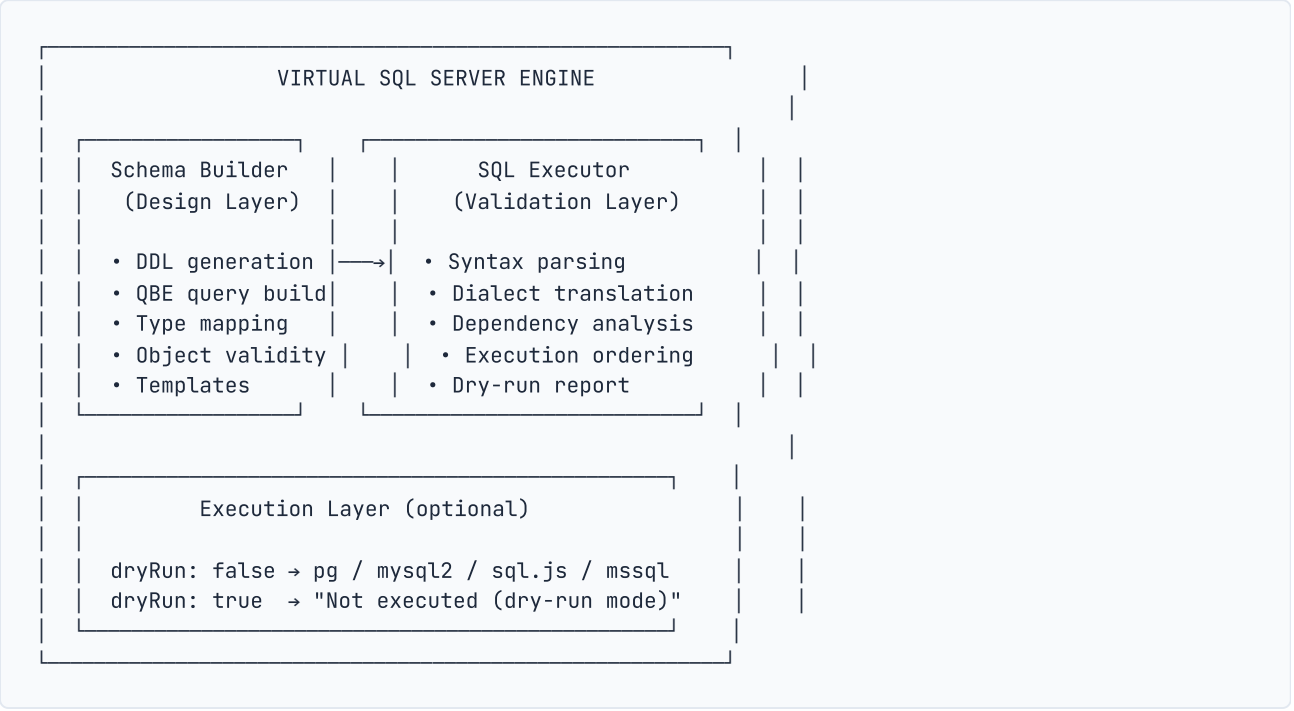
Traditional database tooling operates in two modes:

Mode	Behavior	Pain Points
Connected	Design + generate + execute against a live DB	Requires credentials, network, running server; risk of destructive operations on production data; iteration slow due to connection round-trips
File-based	Write SQL in a text editor, then run via CLI	No validation, no type checking, no dialect translation, no dependency analysis until execution

VSSE fills the gap: a **zero-connection design-and-validation pipeline** that gives the user full confidence their SQL will work, without ever touching a database server.

---

3. Architecture



3.1 Design Layer (Schema Builder)

Zero database imports. Zero network I/O. Zero async.

All design-layer operations run synchronously on pure JavaScript objects:

Component	Input	Output	DB Required
`ddl-generator.js`	Design object with columns, constraints, types	SQL string: `CREATE TABLE`, `CREATE VIEW`, etc.	Never
`query-builder` (QBE)	QBE state: tables, columns, WHERE, joins, unions	SQL string: `SELECT`, `INSERT`, `UPDATE`, `DELETE`	Never
`translator.js`	Column type + engine name	Engine-specific type string (e.g. `VARCHAR(255)` → PostgreSQL `character varying(255)`)	Never
`designer.js`	Raw design input	Validated design object or error	Never
`templates.js`	Type + engine	Boilerplate code for VIEW, FUNCTION, etc.	Never
`config/default.json`	—	Type mappings per engine, categories	Never

The engine is treated as a **configuration parameter** — a key in a lookup table of dialect rules, not a live connection.

3.2 Validation Layer (SQL Executor)

The SQL Executor can run in two modes, and the **default mode is offline**:

```
node tools/sql-executor.js --dry-run --engine postgresql schema.sql
```

In this mode:

1. **Parses** SQL into statements (line-by-line or semicolon-delimited)
2. **Classifies** each statement (CREATE TABLE, ALTER, INSERT, etc.)
3. **Resolves dependencies** between statements (topological sort)
4. **Translates dialects** (e.g., MySQL `AUTO_INCREMENT` → PostgreSQL `SERIAL`)
5. **Prints execution order** with original + translated SQL
6. **Exits** — no socket opened, no credentials needed, no server reached

The output explicitly states: `DB connection: NONE`

3.3 Execution Layer (Optional)

When `dryRun: false` (or `--execute` for SQL Executor), the system creates a real database connection via one of four drivers:

- `pg` — PostgreSQL
- `mysql2` — MySQL
- `sql.js` — SQLite (in-memory or file, still local)
- `mssql` — Microsoft SQL Server

This is the **only** mode that requires credentials, network access, or a running server.

---

4. The Dry-Run as a First-Class Design Principle

The crucial architectural insight: **dry-run is not a debugging aid — it is the default control flow.**

In `schema-builder/lib/builder.js`:

```
async function build(stmts, config) {
  const cfg = normalizeConfig(config);
  const builder = BUILDERS[cfg.engine];

  if (cfg.dryRun) {
    return stmts.map(stmt => ({
      statement: stmt,
      status: 'dry-run',
      detail: 'Not executed (dry-run mode)'
    }));
    // ← Returns here. Never reaches BUILDERS[engine].
  }

  return builder(stmts, cfg);
}
```

The `dryRun` check is evaluated **before** the builder is called. The builder — which would create a `pg.Pool`, `mysql2` connection, or `mssql.ConnectionPool` — is never instantiated. The code path is:

```
dryRun: true  → return DDL as text         [zero I/O]
dryRun: false → create driver → connect    [network I/O]
```

In `tools/sql-executor.js`:

```
if (CLI.dryRun) {
  printDryRunReport(statements, metaMap, deps, filteredOrder, CLI.engine)
  continue
  // ← Skips all execution logic, prints report, moves to next file
}
```

The printDryRunReport function is a **pure rendering function** — it takes parsed statements and dependency metadata, formats them into a human-readable report, and exits. No database driver is loaded, no network socket is opened.

5. Why This Is a New Pattern

VSSE is distinct from existing categories:

Category	Requires Live DB?	Generates SQL?	Validates Structure?	Translates Dialects?
SQL IDEs (pgAdmin, DBeaver)	Yes	Limited	At execution only	No
ORMs (Prisma, Sequelize)	Yes (for migrations)	Yes	Schema-only	Limited
Schema comparison tools (pgdiff)	Yes (two DBs)	Yes	At comparison only	No
SQL linters (sqlfluff)	No	No	Syntax only	Partial
VSSE (this paper)	No (optional)	Yes	Structure + dependencies	Full (all 4 engines)

VSSE is to database schema management what a **compiler** is to programming languages:

Concept	Compiler Analogy	VSSE Analogy
Source	Source code (.c, .ts)	Design object / QBE state
Frontend	Lexer + Parser	Designer.validate + DDL generator
Middle-end	Semantic analysis, optimization	Dependency resolution, dialect translation
Backend	Code generation (x86, ARM)	Dialect-specific SQL output
Dry run	`--syntax-only`, `--emit-llvm`	`--dry-run`, `dryRun: true`
Execution	Runtime (CPU)	Database connection (PG, MySQL, etc.)

Like a compiler that lets you check syntax, view IR, and optimize without ever running the program, VSSE lets you design, generate, validate, and translate SQL without ever connecting to a database.

6. Benefits

Benefit	Description
Zero-infrastructure design	Design schemas and build queries without installing PostgreSQL/MySQL/MSSQL or maintaining a test database
Safe iteration	Generate DDL, review, modify, regenerate — no risk of corrupting a live database
CI/CD integration	Validate schema changes in pull requests without database credentials or network access
Multi-engine portability	Design once, generate SQL for all 4 engines, compare outputs, fix portability issues — all offline
Education & onboarding	Learn SQL schema design without needing server access or installation
Shift-left for schemas	Catch type mismatches, missing constraints, circular dependencies before deployment
Deterministic output	Same input → same output, always. No side effects from database state.

7. Limitations

Limitation	Explanation	Mitigation
No runtime validation	Cannot verify that the SQL runs correctly against a real database with real data	Bounded live execution in CI with ephemeral databases
No data-type compatibility detection	Cannot detect that PostgreSQL 12 vs 18 handle a type differently	Engine version parameter in config
No performance analysis	Cannot generate query plans without a live database	Separate EXPLAIN step against a staging DB
No stored procedure execution	Cannot test PL/pgSQL, T-SQL, or MySQL stored procedures	Unit tests with mock frameworks

8. Conclusion

The Virtual SQL Server Engine pattern **decouples schema design from database connectivity**, treating the database as a pluggable output target rather than a required runtime dependency. The Hexagon Web Framework's Schema Builder and SQL Executor implement this pattern at production grade, supporting 4 database engines across 10+ API routes and a CLI, all with first-class offline operation via dry-run mode.

This pattern is to database tooling what the **three-stage compiler** was to programming languages: a separation of concerns that enables safer, faster, and more portable development workflows.